

Chatbot conversazionale su Caravaggio e Caracciolo

Documentazione tecnica dell'architettura

Knowledge Graph (Neo4j) · RAG Text-to-Cypher · LangGraph
LLM locale (Ollama) · Interfaccia vocale e testuale (Gradio)

*Progetto di Natural Language Processing
Corso di Laurea Magistrale*

Manuel Mignogna

Indice

1	Panoramica del sistema	3
1.1	Struttura del progetto	3
2	Stack tecnologico e librerie	4
2.1	Modello linguistico e orchestrazione	4
2.2	Dati, voce e interfaccia	4
2.3	Selezione del dispositivo e del modello	5
3	La pipeline di ingestion	5
3.1	Estrazione: le query SPARQL	5
3.2	Trasformazione: Extractor	6
3.3	Caricamento: Neo4jLoader e lo schema del grafo	6
4	Il sistema conversazionale: grafo di stato LangGraph	6
4.1	Lo stato: DialogState	6
4.2	I nodi del grafo	7
4.3	Il routing condizionale	7
4.4	Human-in-the-loop e persistenza	8
5	Classificazione e scomposizione della richiesta	8
5.1	Lo schema di output: ModelResponse	8
5.2	Le tre categorie	8
5.3	Scelte di prompting	9
5.4	Robustezza	9
6	Il modulo RAG: da linguaggio naturale a Cypher	9
6.1	Configurazione della catena	9
6.2	Progettazione del prompt Cypher	10
6.3	Esecuzione parallela e resilienza	10
7	Sintesi della risposta	10
8	Interfaccia e canale vocale	11
8.1	Interfaccia Gradio	11
8.2	Voce: Whisper e gTTS	11
9	Sintesi delle scelte architetturali	11

1 Panoramica del sistema

Il progetto realizza un assistente conversazionale specializzato su un dominio ristretto e ben definito: le opere dei pittori **Michelangelo Merisi da Caravaggio** e **Giovanni Battista Caracciolo** (detto Battistello) e i musei di Napoli che le espongono. L'utente può porre domande sia a voce sia per iscritto, e il sistema risponde con testo e sintesi vocale, attingendo a un grafo di conoscenza costruito a partire da fonti aperte (Wikidata e Wikipedia).

L'architettura si articola in **due sottosistemi indipendenti**, separati nel tempo e nelle responsabilità:

1. **Pipeline di ingestion (offline)**. Interroga Wikidata via SPARQL, arricchisce i dati con le descrizioni testuali di Wikipedia e materializza il tutto in un knowledge graph Neo4j. Viene eseguita una tantum (o quando si vuole rigenerare la base dati) tramite `build_database.py`.
2. **Sistema conversazionale (runtime)**. All'avvio (`app.py`) espone un'interfaccia Gradio. Ogni turno di dialogo passa attraverso un grafo di stato LangGraph che classifica la richiesta, la scompone in sotto-domande, recupera i dati dal grafo tramite RAG Text-to-Cypher e sintetizza una risposta in linguaggio naturale, infine convertita in voce.

Il principio di fondo è la **separazione tra conoscenza e ragionamento**: i fatti risiedono in modo strutturato nel grafo, mentre l'LLM è usato come traduttore (linguaggio naturale → Cypher) e come sintetizzatore (dati → risposta), non come fonte di verità. Questo riduce le allucinazioni e ancora le risposte a dati verificabili.

Flusso end-to-end di un turno

Voce/testo utente → (Whisper STT) → classificazione e scomposizione (LLM) → per ogni sotto-domanda in ambito: generazione Cypher (LLM) → query su Neo4j → righe grezze → sintesi unica della risposta (LLM) → (gTTS) → testo + audio all'utente.

1.1 Struttura del progetto

Il codice è organizzato in package Python per responsabilità:

Package / file	Responsabilità
<code>chatbot/ingestion/</code>	Costruzione del knowledge graph: query SPARQL, estrazione, caricamento in Neo4j.
<code>chatbot/rag/</code>	Catena RAG Text-to-Cypher su Neo4j e prompt di generazione query / sintesi.
<code>chatbot/dialog/</code>	Gestione del dialogo: grafo di stato LangGraph, classificazione, stati, prompt di routing.
<code>chatbot/ui/</code>	Interfaccia Gradio e stato di sessione.
<code>chatbot/speech/</code>	Riconoscimento vocale (Whisper) e sintesi vocale (gTTS).
<code>chatbot/config.py</code>	Configurazione centralizzata: credenziali, modelli, percorsi, costanti.
<code>query/*.psql</code>	Le quattro query SPARQL parametriche (artisti, opere, musei).
<code>app.py</code> , <code>build_database.py</code>	Entry point: avvio UI e popolamento del database.

2 Stack tecnologico e librerie

La scelta delle librerie riflette due priorità: eseguire tutto **in locale** (nessuna dipendenza da API a pagamento per l'LLM) e appoggiarsi ad **astrazioni consolidate** (LangChain/LangGraph) per non reimplementare orchestrazione e integrazione con il grafo.

2.1 Modello linguistico e orchestrazione

Libreria	Ruolo	Motivazione della scelta
<code>langchain-ollama</code>	LLM locale (ChatOllama)	Esegue Gemma via Ollama senza costi né chiavi API; espone <code>with_structured_output</code> per l'output tipizzato.
<code>langgraph</code>	Grafo di stato del dialogo	Modella il turno come macchina a stati con nodi, edge condizionali e human-in-the-loop (<code>interrupt</code>).
<code>langgraph-checkpoint-sqlite</code>	Persistenza dello stato	Checkpoint per conservare lo stato di conversazione tra i turni, indicizzato per <code>thread_id</code> .
<code>langchain-neo4j</code>	RAG sul grafo	Fornisce <code>Neo4jGraph</code> e <code>GraphCypherQAChain</code> (Text-to-Cypher) pronti all'uso.
<code>pydantic</code>	Modelli tipizzati	Definisce lo schema dell'output del classificatore e lo stato del dialogo con validazione automatica.

2.2 Dati, voce e interfaccia

Libreria	Ruolo	Motivazione della scelta
<code>sparqlwrapper</code>	Client SPARQL	Interroga l'endpoint di Wikidata; gestione di formato JSON e header.
<code>neo4j</code>	Driver del grafo	Accesso diretto a Neo4j per la fase di caricamento (MERGE dei nodi/relazioni).
<code>transformers</code> + <code>torch</code> + <code>torchaudio</code>	Speech-to-Text	Eseguono Whisper large-v3 in locale per la trascrizione dell'audio.
<code>gtts</code>	Text-to-Speech	Sintesi vocale semplice e in italiano della risposta finale.

<code>gradio</code>	Interfaccia web	UI chat con supporto nativo a microfono, audio e animazioni di caricamento.
<code>mlx-lm</code>	Backend Apple Silicon	Abilita l'inferenza ottimizzata su GPU Apple (variante <code>-mlx</code> del modello).

2.3 Selezione del dispositivo e del modello

La configurazione (`config.py`) sceglie automaticamente l'acceleratore disponibile — CUDA (NVIDIA/Colab), altrimenti MPS (Apple Silicon), altrimenti CPU — e di conseguenza la variante del modello Gemma (`gemma4:e4b-mlx` su Apple, `gemma:e4b` altrove). Tutte le costanti sensibili (URI e credenziali Neo4j, nomi dei modelli, endpoint) sono lette da variabili d'ambiente con valori di default, così da non essere disseminate nel codice.

3 La pipeline di ingestion

L'obiettivo di questa fase è trasformare dati eterogenei e semi-strutturati (Wikidata) e testi discorsivi (Wikipedia) in un grafo pulito e interrogabile. È orchestrata da `pipeline.populate_database()` e si compone di tre stadi: **estrazione**, **trasformazione/arricchimento**, **caricamento** (un classico ETL).

3.1 Estrazione: le query SPARQL

Le quattro query in `query/*.psql` interrogano Wikidata a partire dagli identificatori dei due artisti (`Q42207` per Caravaggio, `Q2519261` per Caracciolo). Sono state scritte con alcune accortezze ricorrenti:

- **Preferenza linguistica con fallback.** Le etichette sono richieste in italiano e, se assenti, in inglese, tramite `COALESCE(?labelIT, ?labelEN, "default")`. Questo evita campi vuoti quando Wikidata non ha la traduzione italiana.
- **Aggregazione per evitare duplicati.** Proprietà multi-valore (soggetti, movimenti, opere notevoli) sono raccolte con `GROUP_CONCAT(DISTINCT ...)`; i valori singoli con `SAMPLE(...)`, così ogni entità produce una sola riga.
- **Fonti alternative unite con UNION.** Il luogo di un'opera è cercato sia come «collocazione» (`P276`) sia come «collezione» (`P195`); l'indirizzo del museo attraverso più proprietà. In questo modo si massimizza la copertura di un grafo di conoscenza notoriamente irregolare.

`SparqlExecutor` carica le query, le esegue con **retry esponenziale sul rate limit** (HTTP 429, attesa e nuovo tentativo) e **memorizza i risultati in cache** (`cache/sparql_cache.json`). La cache è cruciale: senza di essa ogni ricostruzione del database rifarebbe decine di richieste a Wikidata, lente e soggette a throttling.

Nota sulla cache

La cache SPARQL contiene i dati Wikidata (date, dimensioni, relazioni); la cache Wikipedia contiene solo le descrizioni testuali. Modificando una query SPARQL occorre invalidare `sparql_cache.json`, altrimenti la pipeline continua a leggere i risultati vecchi.

3.2 Trasformazione: **Extractor**

Extractor converte le *binding* SPARQL grezze in dizionari Python puliti e le arricchisce:

- Estrae il Q-ID dagli URI Wikidata, tronca le date alla parte utile (anno o data ISO), e traduce le coordinate dal formato WKT `Point(lon lat)` alla coppia `(lat, lon)`.
- Per le opere, **sostituisce la descrizione sintetica di Wikidata con l'introduzione discorsiva della relativa voce Wikipedia**, recuperata in batch dalle API di Wikipedia IT (`prop=extracts`, solo intro, testo semplice). Anche qui interviene una cache dedicata (`wikipedia_cache.json`) con gestione di redirect e normalizzazioni dei titoli, e throttling per rispettare i limiti del servizio.

Questa scelta è deliberata: le descrizioni di Wikipedia sono molto più ricche e utili per la risposta finale rispetto alle stringhe minimali di Wikidata.

3.3 Caricamento: **Neo4jLoader** e lo schema del grafo

Neo4jLoader è implementato come **context manager** (`__enter__` / `__exit__`) per garantire la chiusura del driver Neo4j anche in presenza di eccezioni. Ogni scrittura avviene in una sessione gestita con `with`, e le query Cypher sono tipizzate come `LiteralString` — una scelta di sicurezza che segnala staticamente l'intenzione di non costruire query da input dinamico.

I nodi sono inseriti con `MERGE ... ON CREATE SET`, idempotente: rieseguire il caricamento non duplica i nodi. Lo schema risultante è il seguente:

Nodo	Proprietà principali
Artista	<code>name</code> , <code>data_nascita</code> , <code>data_morte</code> , <code>luogo_nascita</code> , <code>movimenti</code> , <code>opere_notevoli</code> , <code>wikidataId</code>
Opera	<code>name</code> , <code>anno</code> , <code>altezza</code> , <code>larghezza</code> , <code>tecnica</code> , <code>soggetti</code> , <code>descrizione</code> , <code>tipo</code> , <code>wikidataId</code>
Museo	<code>name</code> , <code>descrizione</code> , <code>indirizzo</code> , <code>sito</code> , <code>telefono</code> , <code>fondazione</code> , <code>latitudine</code> , <code>longitudine</code> , <code>biglietto</code>
Città	<code>name</code>

Le relazioni collegano le entità secondo la semantica del dominio:

```
(Opera) - [:DIPINTA_DA] -> (Artista)
(Opera) - [:ESPOSTA_IN] -> (Museo)
(Museo) - [:SITUATO_IN] -> (Città)
```

L'ordine di caricamento (artisti e musei prima delle opere) non è casuale: `insert_work` esegue un `MATCH` su artista e museo per creare le relazioni, quindi quei nodi devono già esistere.

4 Il sistema conversazionale: grafo di stato LangGraph

Il cuore del runtime è **DialogManager**, che modella un turno di dialogo come un **grafo di stato** (`StateGraph`) di LangGraph. Questa scelta permette di rappresentare esplicitamente il flusso decisionale — classificare, eventualmente chiedere chiarimenti, recuperare e rispondere — con edge condizionali invece che con una cascata di `if` annidati.

4.1 Lo stato: **DialogState**

Lo stato che attraversa il grafo è un modello Pydantic con i campi essenziali del turno:

Campo	Significato
<code>question</code>	La domanda corrente dell'utente (eventualmente arricchita col chiarimento).
<code>sub_questions</code>	Elenco di sotto-domande atomiche, ciascuna con un flag <code>in_scope</code> .
<code>clarification_question</code>	La domanda di chiarimento da porre all'utente, se necessaria.
<code>clarification_attempts</code>	Contatore dei tentativi di chiarimento (per evitare loop infiniti).
<code>history</code>	Gli ultimi turni della conversazione, come lista di <code>Turn</code> (domanda + risposta).
<code>response</code>	La risposta finale (usata anche per il chitchat).

Gli aggiornamenti allo stato usano un `TypedDict` parziale (`DialogStateUpdate`): ogni nodo restituisce solo i campi che modifica, e LangGraph li fonde nello stato. La cronologia è mantenuta **limitata** (ultimi 10 turni salvati, ultimi 3 passati come contesto al classificatore) per contenere la dimensione del prompt.

4.2 I nodi del grafo

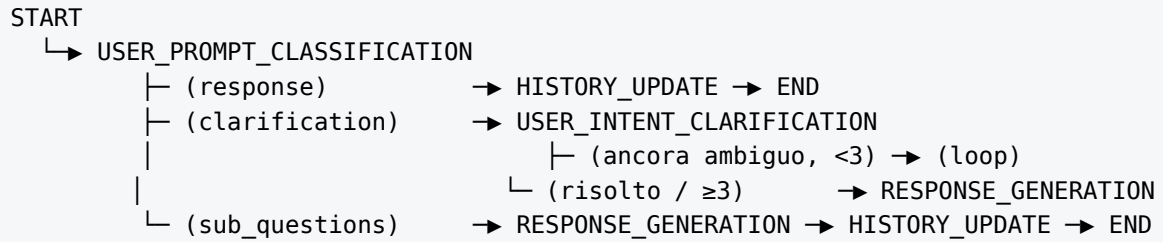
Nodo	Funzione
<code>USER_PROMPT_CLASSIFICATION</code>	Classifica e scompone la richiesta chiamando l'LLM con output strutturato.
<code>USER_INTENT_CLARIFICATION</code>	Sospende il grafo (<code>interrupt</code>) e chiede un chiarimento all'utente.
<code>RESPONSE_GENERATION</code>	Esegue il RAG sulle sotto-domande in ambito e sintetizza la risposta.
<code>HISTORY_UPDATE</code>	Accoda il turno corrente alla cronologia e termina.

4.3 Il routing condizionale

Dopo la classificazione, una funzione di routing (`_route_after_resolve`) decide il nodo successivo in base a **quali campi dello stato sono valorizzati**, con una precisa priorità:

1. se è presente una `response` (caso chitchat) → si va direttamente all'aggiornamento cronologia, saltando del tutto il grafo;
2. se è presente una `clarification_question` → si va al nodo di chiarimento;
3. se sono presenti `sub_questions` → si va alla generazione della risposta.

Il nodo di chiarimento ha a sua volta un edge condizionale: se dopo la risposta dell'utente la richiesta è ancora ambigua e non si sono superati i **3 tentativi**, si richiede un nuovo chiarimento; altrimenti si procede comunque alla generazione. Questo limite evita che l'utente resti intrappolato in un ciclo di domande.



Codice 1: Flusso del grafo di stato del dialogo.

4.4 Human-in-the-loop e persistenza

Il chiarimento sfrutta il meccanismo `interrupt` di LangGraph: il grafo si **sospende** restituendo la domanda all'interfaccia e riprende esattamente da quel punto quando arriva la risposta dell'utente (`Command(resume=...)`). Perché queste funzioni tra due invocazioni HTTP distinte serve un **checkpoint**: lo stato è serializzato e indicizzato per `thread_id`, cioè per conversazione. Ogni sessione Gradio genera un `thread_id` (UUID) nuovo, così conversazioni diverse non si mescolano.

5 Classificazione e scomposizione della richiesta

Il primo nodo è anche il più delicato: deve decidere **cosa** fare con il messaggio dell'utente prima ancora di toccare il grafo. Usa l'LLM in modalità **output strutturato** (`with_structured_output(ModelResponse)`), che vincola la risposta del modello allo schema Pydantic.

5.1 Lo schema di output: `ModelResponse`

```

class ModelResponse(BaseModel):
    type: Literal["query", "clarification", "chitchat"]
    sub_questions: Optional[list[SubQuestion]] = None # se type == query
    clarification_question: Optional[str] = None # se type == clarification
    response: Optional[str] = None # se type == chitchat

```

Ogni tipo usa **un solo campo payload**, senza combinazioni ambigue: questo mantiene la logica di routing pulita (uno switch sul `type` decide quale campo leggere). È stato preferito ad avere un campo separato per il chitchat proprio per non introdurre stati ridondanti da disambiguare.

5.2 Le tre categorie

Categoria	Quando scatta e cosa produce
QUERY	La richiesta contiene almeno una domanda su opere/artisti/musei. Viene scomposta in sotto-domande atomiche, ciascuna auto-contenuta e con un flag <code>in_scope</code> .
CLARIFICATION	La richiesta richiederebbe una query ma contiene un riferimento implicito che né il messaggio né la storia risolvono (es. «Chi l'ha dipinta?» senza opera nominata).
CHITCHAT	Messaggi puramente sociali (saluti, ringraziamenti) senza alcuna richiesta di informazioni. La risposta cordiale è generata direttamente nel campo <code>response</code> .

5.3 Scelte di prompting

Il prompt del classificatore concentra diverse istruzioni non banali:

- **Risoluzione dei riferimenti (coreference).** Ogni sotto-domanda deve essere riscritta in forma completamente auto-contenuta: dimostrativi e pronomi («questi quadri», «lui», «lì») vanno sostituiti con il nome esplicito dell'entità, preso dagli scambi precedenti. Chi legge la sotto-domanda non deve aver bisogno della conversazione per capirla — requisito essenziale perché la generazione Cypher a valle riceve le sotto-domande isolate.
- **Scomposizione di richieste composte.** Una domanda che ne contiene più di una viene spezzata in unità atomiche su un solo argomento. Ciò consente di recuperare e poi ricombinare i dati in modo controllato.
- **Marcatura dell'ambito (`in_scope`).** Ogni sotto-domanda è etichettata come dentro o fuori dominio (Caravaggio/Caracciolo, loro opere, musei di Napoli). Le parti fuori ambito non vengono interrogate sul grafo ma gestite con una nota esplicita, così una richiesta mista («...e quante ne ha fatte Botticelli?») viene comunque servita per la parte pertinente.
- **Bias verso QUERY in caso di dubbio,** per non rifiutare domande legittime, e disambiguazione esplicita tra i due artisti.

Coerenza tra prompt e schema

Poiché l'output è vincolato allo schema Pydantic, le chiavi indicate negli esempi del prompt devono coincidere esattamente con i nomi dei campi (`response` , `clarification_question` , `sub_questions`). Un disallineamento — per esempio suggerire una chiave `text` inesistente nello schema — porta il modello a produrre un campo che viene scartato, lasciando il valore atteso a `None` .

5.4 Robustezza

Con modelli locali di piccola taglia l'output strutturato può occasionalmente fallire (stringa vuota o JSON malformato). È previsto — o comunque consigliato — un **fallback difensivo**: intercettare l'errore di parsing e degradare verso una richiesta di chiarimento, invece di far crashare l'intero turno. La `temperature` bassa (0 per la classificazione) riduce la variabilità e rende l'output più deterministico.

6 Il modulo RAG: da linguaggio naturale a Cypher

Il recupero delle informazioni segue il paradigma **RAG Text-to-Cypher**: invece di cercare in un indice vettoriale, l'LLM traduce la domanda in una query Cypher che viene eseguita sul grafo Neo4j. `RagChain` incapsula la connessione al grafo, l'LLM e la `GraphCypherQChain` di `LangChain`.

6.1 Configurazione della catena

```
GraphCypherQChain.from_llm(  
    llm=self.llm,  
    graph=Neo4jGraph(...),  
    cypher_prompt=CYPHER_PROMPT,  
    return_direct=True,      # restituisce le righe grezze, salta la sintesi interna  
    validate_cypher=True,    # corregge la direzione delle relazioni secondo lo  
    schema  
    allow_dangerous_requests=True,  
)
```

Due scelte meritano una spiegazione:

- **return_direct=True**. Di norma `GraphCypherQChain` fa due chiamate all'LLM: una per generare il Cypher, una per trasformare i risultati in prosa. Qui la seconda è disattivata: la catena è usata **solo per il retrieval** e restituisce le righe grezze. La trasformazione in risposta discorsiva è centralizzata altrove, in **un'unica** chiamata di sintesi per l'intero turno. Così una domanda composta da tre sotto-domande costa una sola sintesi invece di tre, risparmiando chiamate.
- **validate_cypher=True**. Corregge automaticamente le direzioni delle frecce nelle relazioni per farle combaciare con lo schema, evitando che un arco orientato al contrario restituisca silenziosamente un risultato vuoto.

Lo **schema del grafo viene iniettato automaticamente** nel prompt da `Neo4jGraph`, che lo introspetta: il modello conosce quindi etichette, proprietà e relazioni disponibili quando genera la query.

6.2 Progettazione del prompt Cypher

Il prompt di generazione (`CYPHER_GENERATION_TEMPLATE`) codifica una serie di regole nate dagli errori tipici dei modelli piccoli su Text-to-Cypher:

- **Relazioni bidirezionali senza frecce**. Si impone la sintassi piatta `-[:REL]-` senza `</>`. Un modello che indovina la direzione sbagliata produrrebbe risultati vuoti; togliendo l'orientamento il match funziona a prescindere.
- **Case-insensitive**. Confronti sempre con `toLower(...)` su entrambi i lati, per non fallire su differenze di maiuscole.
- **Ricerca per contenimento**. Musei e opere sono cercati con `CONTAINS` sul nome anziché per uguaglianza esatta, per tollerare titoli parziali («Flagellazione», «Capodimonte»).
- **Disambiguazione degli artisti**. Istruzioni esplicite per non confondere Caravaggio (`Merisi`) e Caracciolo (`Battistello`), che restano due nodi distinti.
- **Alias obbligatori sugli aggregati**. Ogni `count / collect / sum` deve avere un `AS` leggibile, così le righe restituite hanno chiavi comprensibili per la sintesi.
- **Gestione di entità multiple**. Quando la domanda riguarda più opere, si impone un unico pattern con `WHERE ... IN [...]` restituendo ogni entità col proprio valore, anziché legarle tutte allo stesso nodo (errore che spesso produce risultati vuoti).

6.3 Esecuzione parallela e resilienza

Nel nodo di generazione, le sotto-domande **in ambito** sono deduplicate e interrogate **in parallelo** con un `ThreadPoolExecutor` (fino a 5 worker): poiché ogni query è indipendente e passa gran parte del tempo in attesa di rete/DB, il parallelismo abbatte la latenza del turno. Ogni query ha inoltre una **politica di retry** (fino a 3 tentativi) sui risultati vuoti o sulle eccezioni, per assorbire la variabilità del Cypher generato dall'LLM.

7 Sintesi della risposta

Recuperate le righe dal grafo, un'**unica** chiamata all'LLM (`COMBINE_PROMPTS_TEMPLATE`) le fonde in una risposta coerente in italiano. Il prompt di sintesi è fortemente vincolato nello stile, per contrastare le tendenze indesiderate dei modelli generativi:

- risposta **concisa** (3–4 frasi), diretta al dato, senza premesse né inviti finali;
- **divieto di citare la fonte** («secondo il database», «nel mio archivio»): il modello deve rispondere come se conoscesse i fatti;
- **divieto di disclaimer** quando i dati ci sono, e uso di **tutti e soli** i valori presenti (se è una lista, elencarli tutti senza inventarne);

- se i dati di una sotto-domanda sono vuoti, quella informazione è dichiarata non disponibile senza inventare, rispondendo comunque alle altre.

Alle sotto-domande **fuori ambito** è riservata una nota fissa che ricorda i confini del dominio, concatenata alla risposta in-ambito. In questo modo una richiesta mista riceve una risposta completa e onesta: dati reali dove il grafo li ha, delimitazione esplicita dove no.

Perché sintesi separata dal retrieval

Separare il recupero (una query per sotto-domanda) dalla sintesi (una sola chiamata per turno) è la scelta architetturale che tiene basso il numero di invocazioni dell'LLM e centralizza il controllo dello stile in un unico prompt, invece di disperderlo nella generazione interna della catena.

8 Interfaccia e canale vocale

8.1 Interfaccia Gradio

`ChatController` costruisce l'interfaccia e ne espone gli handler. Ogni turno è diviso in **due eventi concatenati** (`step 1 .then(step 2)`): il primo aggiunge subito il messaggio dell'utente alla chat (feedback immediato), il secondo calcola la risposta mostrando l'animazione di caricamento nativa. Sono supportati due ingressi — testo e microfono — che convergono sullo stesso passo di generazione.

Lo stato di sessione (`SessionState` , in `gr.State`) conserva due informazioni: il `thread_id` della conversazione e un flag `awaiting_clarification` , che indica se il messaggio successivo va instradato come **risposta a un chiarimento** anziché come nuova domanda. Il pulsante «Nuova conversazione» rigenera il `thread_id` , ripulendo di fatto la storia.

8.2 Voce: Whisper e gTTS

Il riconoscimento vocale (`SpeechToText`) usa **Whisper large-v3** tramite la pipeline di `transformers` , forzando la lingua italiana e adattando il formato audio (`sample_rate, ndarray`) prodotto dal microfono di Gradio. La sintesi vocale (`TextToSpeech`) usa **gTTS** per generare l'mp3 della risposta. La sintesi è **tollerante ai guasti**: se il servizio TTS fallisce, il turno mostra comunque la risposta testuale, semplicemente senza audio.

9 Sintesi delle scelte architetturali

Scelta	Motivazione
Knowledge graph invece di RAG vettoriale	Il dominio è fattuale e relazionale (chi ha dipinto cosa, esposta dove): un grafo risponde con precisione a conteggi e relazioni, dove l'embedding sarebbe approssimativo.
LLM come traduttore/sintetizzatore	Ancora le risposte a dati verificabili nel grafo, riducendo le allucinazioni; il modello non è la fonte di verità.
LLM locale via Ollama	Nessun costo per token né chiave API; esecuzione interamente in locale, adatta a un progetto didattico.

Grafo di stato LangGraph	Rende esplicito e manutenibile il flusso decisionale del turno, con chiarimenti human-in-the-loop e persistenza per conversazione.
Output strutturato Pydantic	Trasforma la classificazione in un contratto tipizzato, eliminando il parsing fragile di testo libero.
Scomposizione + risoluzione riferimenti	Permette di gestire domande composte e conversazionali, isolando sotto-domande auto-contenute per il retrieval.
Retrieval parallelo + sintesi unica	Minimizza latenza e numero di chiamate LLM, centralizzando lo stile in un solo prompt.
Cache SPARQL e Wikipedia	Rende la ricostruzione del database rapida e rispettosa dei rate limit delle fonti.
Degradazione elegante (voce, retry, fallback)	Il sistema resta utilizzabile anche quando un componente esterno o l'LLM falliscono transitoriamente.

Documento generato come descrizione tecnica dell'architettura del progetto.